

We write the output of this classifier as:

$$p(y|\mathbf{h}^{(i)}) \propto \exp(\mathbf{h}^{(i)} \cdot \mathbf{y} + \beta_y) \quad (4)$$

where $\mathbf{y}$ and β_y are the learned weight and bias terms for the label y , and i represents the index of the [MASK] token.

In the second step, we substitute $\mathbf{h}^{(i)}$ with the MLM’s output word embeddings $\mathbf{w}_{\text{out}}$ to obtain a score $s(y, w) = p(y|\mathbf{w}_{\text{out}})$. Intuitively, because $\mathbf{w}_{\text{out}} \cdot \mathbf{h}$ and $\mathbf{y} \cdot \mathbf{h}$ are large for words and labels that are relevant to a particular context, $s_w \propto \exp(\mathbf{w}_{\text{out}} \cdot \mathbf{y} + \beta_y)$ should be large for words that are typically associated with a given label. The sets of label tokens are then constructed from the k -highest scoring words:

$$\mathcal{V}_y = \underset{w \in \mathcal{V}}{\text{top-}k} [s(y, w)] \quad (5)$$

2.4 Relation to Other Prompting Methods

Our work fits into a body of work that probes language model’s knowledge via prompts. Previous works have used manually defined prompts to study an LM’s ability to perform: commonsense reasoning (Trinh and Le, 2018; Kwon et al., 2019; Shwartz et al., 2020), question answering (Lewis et al., 2019), fact recall (Petroni et al., 2019; Jiang et al., 2020; Bouraoui et al., 2019), summarization (Radford et al., 2019), and other supervised tasks (Brown et al., 2020). Schick and Schütze (2020) use manually constructed prompts in conjunction with semi-supervised learning for few-shot learning. We instead *automatically* create prompts for any task, which leads to higher accuracy and opens up new phenomena to analyze.

2.5 Evaluation Setup

In the following sections, we apply AUTOPROMPT to probe BERT_{BASE}² (110M parameters) and RoBERTa_{LARGE}’s (355M parameters) knowledge of the following tasks: sentiment analysis, natural language inference (NLI), fact retrieval, and relation extraction. We use the PyTorch implementations and pretrained weights provided by the transformers Python library (Wolf et al., 2019). For sentiment analysis and NLI, we find label tokens using the logistic-regression-based heuristic described in Section 2.3. For fact retrieval and relation extraction, we skip this step as the labels (entities) directly correspond to tokens in the vocabulary. For all tasks, we perform the prompt

²For brevity, we will omit subscripts in the model names.

search described in Section 2.2 for multiple iterations. In each iteration, we use a batch of training data to identify the candidate set $\mathcal{V}_{\text{cand}}$ of replacement trigger tokens. We then evaluate the label likelihoods of the updated prompts on a separate batch of data, and we retain the best trigger token in the next iteration of the search. At the end of every iteration, we measure the label likelihood on withheld development data, and return the best prompt found during the entire search as the final output. Performance is evaluated using the appropriate task-specific metrics—e.g., accuracy for sentiment analysis and NLI, and precision@ k for fact retrieval—on a separate withheld test set.

Our AUTOPROMPT implementation is publicly available at <http://ucinlp.github.io/autoprompt>, and supports prompt generation for pretrained models in the HuggingFace transformers library (Wolf et al., 2019) on arbitrary datasets.

3 Sentiment Analysis

Sentiment analysis is a fundamental task in NLP, both for natural language understanding research and real-world applications. It is also difficult to probe the extent to which MLMs understand sentiment without finetuning.

Setup We apply our method to convert instances from the binary Stanford Sentiment Treebank (Socher et al., 2013, SST-2) into prompts, using the standard train/test splits. We find label tokens using a prompt based on the template in Table 3. For our gradient-based prompt search, we perform a grid search over the following hyperparameters: $|\mathcal{V}_{\text{cand}}| \in \{10, 100\}$, $|\mathcal{V}_y| \in \{1, 3, 5\}$, $|\mathbf{x}_{\text{trig}}| \in [3, 6]$.³ All prompts are initialized with the same template used to find the label set.

We also construct a prompt manually (before automated prompts are generated, to avoid bias) based on the intuition that SST-2 is comprised of movie reviews. We use “{sentence} this movie was [P].” as the template, and use “terrible” and “fantastic” for the negative and positive label tokens, respectively.

Results We show results in Table 1, along with reference scores from the GLUE (Wang et al., 2019) SST-2 leaderboard, and scores for a linear probe trained over the elementwise average of the LM token representations. Prompts generated by AUTOPROMPT reveal that both BERT

³Required 2 days to run with 8 NVIDIA 2080Ti GPUs.

Model	Dev	Test
BiLSTM	-	82.8 [†]
BiLSTM + ELMo	-	89.3 [†]
BERT (linear probing)	85.2	83.4
BERT (finetuned)	-	93.5 [†]
RoBERTa (linear probing)	87.9	88.8
RoBERTa (finetuned)	-	96.7 [†]
BERT (manual)	63.2	63.2
BERT (AUTOPROMPT)	80.9	82.3
RoBERTa (manual)	85.3	85.2
RoBERTa (AUTOPROMPT)	91.2	91.4

Table 1: **Sentiment Analysis** performance on the SST-2 test set of supervised classifiers (top) and fill-in-the-blank MLMs (bottom). Scores marked with [†] are from the GLUE leaderboard: <http://gluebenchmark.com/leaderboard>.

and RoBERTa have a strong knowledge of sentiment analysis: without any finetuning, BERT performs comparably to a supervised BiLSTM, and RoBERTa achieves an accuracy on-par with finetuned BERT and ELMo models. In addition, we observe that our automatically constructed prompts are more effective than manual prompts, and that they are difficult to construct using human intuition: the best template for RoBERTa is “{sentence} atmosphere alot dialogue Clone totally [P].” We include results on the effect of the AUTOPROMPT hyperparameters in Appendix A.

Accuracy in Low-Data Settings Although the goal of AUTOPROMPT is to probe a model’s knowledge, we also find that it may be a viable alternative to finetuning in the low-data regime. To show this, we measure the development set accuracy of AUTOPROMPT prompts when using random subsets of 10, 100, and 1000 instances from the training data. We run our prompt search with $|x_{\text{trig}}| = 10$, $|\mathcal{V}_y| = 3$, and $|\mathcal{V}_{\text{cand}}| = 10$. We compare to the performance of BERT and RoBERTa finetuned on the same data. For fair comparison between AUTOPROMPT and finetuning, we use Mosbach et al. (2020)’s recommended parameters for finetuning on small datasets: trained for 20 epochs, using AdamW (Loshchilov and Hutter, 2018) with bias correction and a learning rate that linearly increases to 2×10^{-5} in the first 10% of iterations, and linearly decreases to 0 afterwards. Experiments are repeated 10 times on random subsets of data (and seeds for the finetuned models). Best-case, worst-case, and average performance are shown in Figure 2. Note that results in the EMNLP version had a bug that has since been fixed.

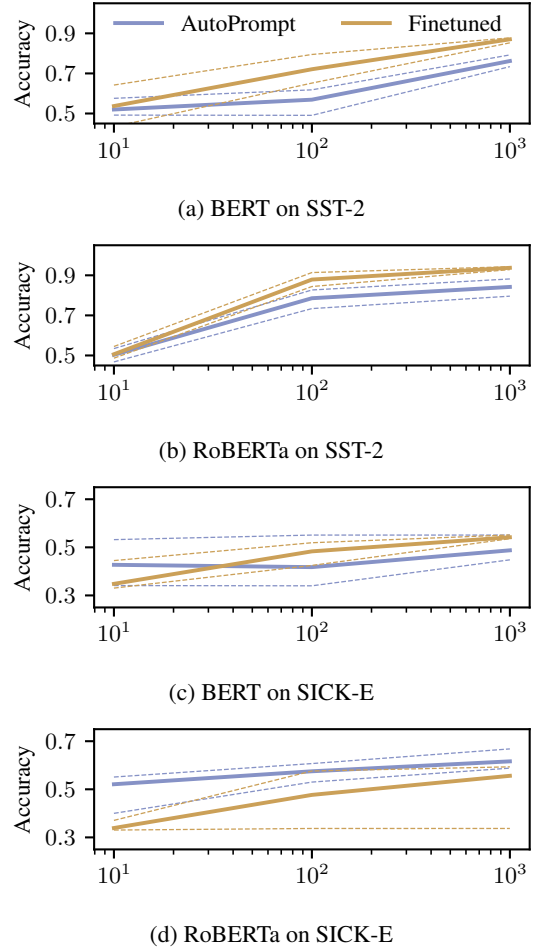


Figure 2: **Effect of Training Data** on sentiment analysis and NLI for AUTOPROMPT vs. finetuning. X-axis is the number of data points used during training. Error bars plot the max. and min. accuracies observed over 10 independent runs. (revised since EMNLP version).

We observe that while finetuning outperforms AUTOPROMPT on sentiment analysis, AUTOPROMPT can perform better than finetuning on NLI. Notably, AUTOPROMPT elicits better average performance from both BERT and RoBERTa given only 10 training examples. Furthermore, results for RoBERTa are more stable across all sample sizes whereas finetuning can result in “failed runs” (consistent with Dodge et al. 2020). This behavior in the low-data regime is an interesting phenomenon, and suggests that there are barriers that MLMs must surmount when they are converted to finetuned classifiers that are not encountered when the task is presented as masked language modeling.

4 Natural Language Inference

To evaluate the *semantic* understanding of MLMs, we experiment on Natural Language Inference

Model	SICK-E Datasets		
	standard	3-way	2-way
Majority	56.7	33.3	50.0
BERT (finetuned)	86.7	84.0	95.6
BERT (linear probing)	68.0	49.5	91.9
RoBERTa (linear probing)	72.6	49.4	91.1
BERT (AUTOPROMPT)	62.3	55.4	85.7
RoBERTa (AUTOPROMPT)	65.0	69.3	87.3

Table 2: **Natural Language Inference** performance on the SICK-E test set and variants. (Top) Baseline classifiers. (Bottom) Fill-in-the-blank MLMs.

(NLI). NLI is crucial in many tasks such as reading comprehension and commonsense reasoning (Bowman et al., 2015), and it is used as a common benchmark for language understanding.

Setup We use the entailment task from the SICK dataset (Marelli et al., 2014, SICK-E) which consists of around 10,000 pairs of human-annotated sentences labeled as entailment, contradiction, and neutral. The standard dataset is biased toward the neutral class which represent 56.7% of instances. We also experiment on an unbiased variant with 2-way classification of contradiction vs. entailment (2-way), as well as an unbiased 3-way classification variant (3-way). The template used for AUTOPROMPT is provided in Table 3. We search over the following parameters: $|\mathcal{V}_{cand}| \in \{10, 50\}$, $|\mathcal{V}_y| \in \{1, 3, 5, 10\}$, $|\mathbf{x}_{trig}| \in [1, 5]$, and choose the best prompt according to development set accuracy.

Results Table 2 shows that AUTOPROMPT considerably outperforms the majority baseline in all experiments. For example, on the 2-way SICK-E dataset, AUTOPROMPT is comparable to a supervised finetuned BERT. We also test linear probes—linear classifiers trained on top of frozen MLM representations with average pooling—and find AUTOPROMPT has comparable or higher accuracy, despite linear probes being susceptible to false positives. Overall, these results demonstrate that both BERT and RoBERTa have some inherent knowledge of natural language inference.

We also examine the efficacy of AUTOPROMPT in the low-data regime (using the same procedure as SST-2) on the unbiased 3-way SICK-E data. The results in Figure 2 show that AUTOPROMPT performs on par with finetuned BERT and significantly better than finetuned RoBERTa in low data settings.

MLMs Excel on Contradiction We find that the label tokens are more interpretable for *con-*

tradiction compared to *entailment* or *neutral* (examples in Table 3). We investigate if this hurts the model performance on entailment and neutral classes. We measure the precision for each label in the 3-way balanced SICK-E dataset. BERT achieves 74.9%, 54.4%, and 36.8% precision for contradiction, entailment, and neutral cases, respectively, while RoBERTa obtains 84.9%, 65.1%, and 57.3%. These results suggest that AUTOPROMPT may be more accurate for concepts that can be easily expressed using natural label tokens.

5 Fact Retrieval

An important question is whether pretrained MLMs *know* facts about real-world entities. The LAMA dataset (Petroni et al., 2019) evaluates this using cloze tests that consist of (sub, rel, obj) triples, e.g. (Obama, bornIn, Hawaii), and *manually* created prompts with missing objects, e.g. “Obama was born in [MASK].”. LPAQA (Jiang et al., 2020) extends this idea by *systematically* creating prompts that are generated by mining Wikipedia, paraphrasing, and crowdsourcing. In this section, we use the same cloze-style setup but *automatically* generate prompts in order to better evaluate the factual knowledge of MLMs. We compare our approach against LAMA and LPAQA, which are explicitly designed for the task of fact retrieval.

Setup We reformulate fact retrieval by mapping (sub,rel,obj) triples to a prompt using the template “{sub}[T]...[T][P].”, where the trigger tokens are specific to the relation rel and the correct object obj is the label token. We use the original test set from LAMA (Petroni et al., 2019), henceforth *Original*. To collect training data for AUTOPROMPT, we gather at most 1000 facts for each of the 41 relations in LAMA from the T-REx dataset (ElSahar et al., 2018). For the relations that still have less than 1000 samples, we gather extra facts straight from Wikidata. We ensure that none of the T-REx triples are present in the test set, and we split the data 80-20 into train and development sets. Moreover, because the collected T-REx data is from a slightly different distribution than the LAMA test set, we also consider a separate evaluation where we split the T-REx triples into a 60-20-20 train/dev/test split and evaluate on the test set. This *T-REx* dataset is used to measure the performance of our prompts when the train and test data is from the same distribution.